Cryo

Cold data finder for ONTAP NAS volumes: how it works, from the GUI to the cluster, including scopes, results storage, logic and security

1. What it is

Cryo (cryo.py) is a single-file Python tool with a browser GUI. It finds files on NetApp ONTAP NAS volumes (NFS and SMB data) that have not been modified or accessed for a number of days that you choose. It reads folder listings through the ONTAP REST API, so nothing is installed on the cluster and File System Analytics does not need to be enabled. It is read-only: every request it sends to ONTAP is an HTTP GET.

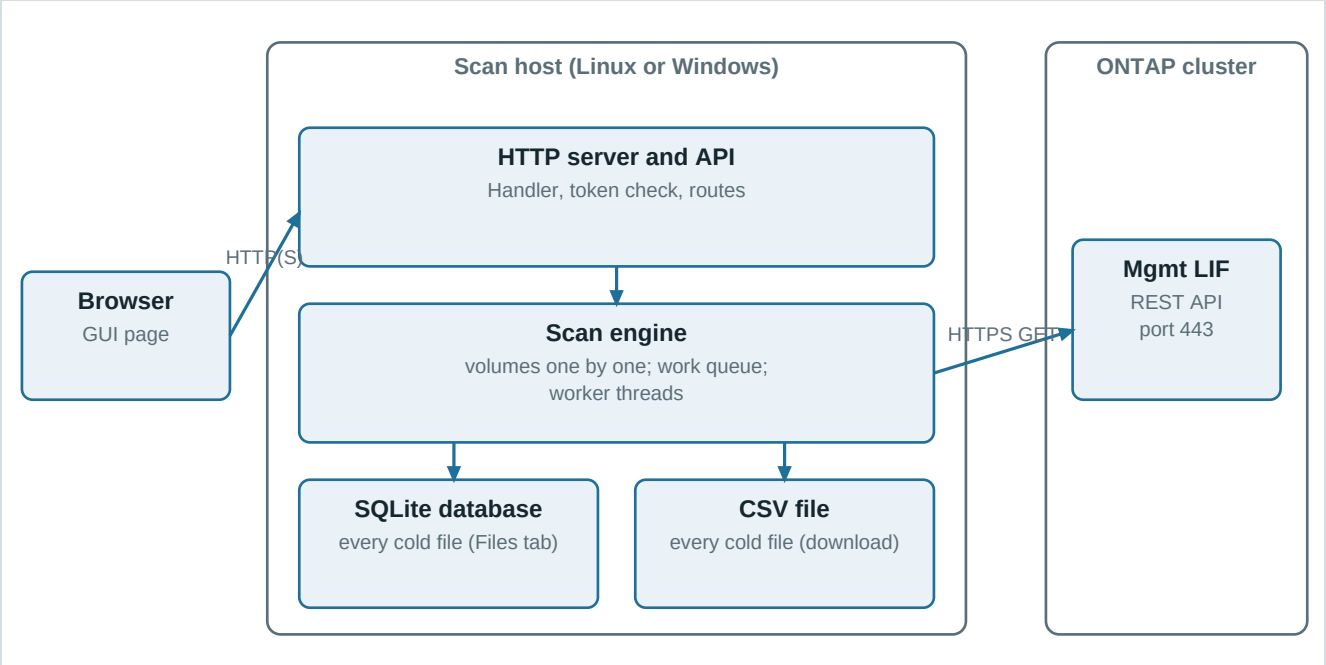
- **No extra installs.** Python 3.6 or newer, standard library only (SQLite comes with Python). No pip, no web framework, no internet access.
- **Air-gap friendly.** The whole GUI (HTML, CSS and JavaScript) is embedded in the script. It loads no fonts, libraries or scripts from outside.
- **One volume, a few, an SVM or the whole cluster.** Volumes are scanned one after another; the results carry the SVM and volume of every file.
- **Credentials stay in memory.** The ONTAP password is never written to disk.
- **Results are saved.** Each scan writes a CSV file (for download) and a SQLite database (for the Files tab). Saved scans can be reopened in the GUI after a restart.

> **Validation status.** The tool has been tested against a mock ONTAP server (one volume and a multi-volume cluster with skipped, offline and failing volumes), against a one-million-row database, and by rendering the GUI in a headless browser. It has not yet been run against a real cluster. Do the first real run on a small volume and a small start folder (see section 9).

Contents

- 2. Architecture
- 3. What can be scanned (scopes)
- 4. From the GUI to the script
- 5. How it connects to ONTAP
- 6. How a scan works
- 7. Results, the database, sorting, filters and the CSV
- 8. Security
- 9. Running it and troubleshooting
- 10. Limits and known gaps
- Appendices: A settings and limits, B glossary, C the Charts tab

2. Architecture



The script has four parts, all in one file.

Part	Where in the script	What it does
Embedded GUI	PAGE (a string of HTML, CSS and JavaScript)	The page the browser shows. Calls the script's API with fetch() and draws the results.
HTTP server	Server, Handler, App	Serves the page and the API. Checks the access token. Validates input. Holds the current scan.
Scan engine	Scan, SavedScan, query_db	Runs a scan in background threads, volume by volume. Keeps counters. Writes the CSV and the SQLite database. Answers sort and filter requests from the database.
ONTAP client	Ontap, skip_reason, quote_path, parse_ts	Sends the REST requests with basic authentication, TLS, retries and paging. Lists volumes and decides which ones the SVM and cluster scopes skip.

Threading model

- The HTTP server creates one thread per browser request, so the GUI stays responsive during a scan.
- A scan runs in its own background thread. For each volume it starts the worker threads (1 to 16, default 6), waits for them to finish, then moves to the next volume.
- Everything the threads share (counters, queue accounting, the CSV writer and the database connection used for writing) is protected by one lock (Scan.lock). Workers hold it only briefly, to add a folder's results. REST calls happen outside the lock.
- The Files tab reads the database through its own short-lived connections. The database is in WAL mode, so reading never blocks the scan that is writing.
- Only one scan can run at a time. Starting a second returns an error until the first is stopped or finished.

3. What can be scanned (scopes)

The ONTAP files API works inside one volume, so a larger scan is simply several volume scans in a row. Each volume is read from its own root, using the same Workers setting. Pick the scope in the Scan panel.

Scope	Volumes scanned	Notes
One volume	The one you pick	The original behaviour. Optional start folder (relative to the volume). Skip rules do not apply.
Selected volumes	The ones you tick in the list	Tick by hand, or use Tick all eligible. Ticked volumes are always scanned, even a DP or offline one.
One SVM	Every eligible volume of the SVM you name	The default skip rules apply (below). The SVM name is not case sensitive.
Whole cluster	Every eligible volume of the cluster	The default skip rules apply (below).

Volumes skipped by default (SVM and cluster scopes)

Skipped	Why	How it is recognised
System volumes	ONTAP's own metadata volumes, not user data	Name starts with MDV_
SVM root volumes	They hold the namespace, not user data	is_svm_root is true, or the volume is mounted at /
FlexGroup constituents	Pieces of a FlexGroup. The FlexGroup itself is scanned instead, so nothing is counted twice	style is flexgroup_constituent
Volumes that are not online	Offline, restricted or mixed volumes cannot be read	state is not online
DP volumes	A SnapMirror (or SnapVault) destination is a copy of another volume. Scanning it counts the same data twice	type is dp (or ls, a load-sharing mirror). The option **Include SnapMirror destination (DP) volumes** turns this one rule off

The Scan panel shows a preview line before you start, for example *6 volumes will be scanned; skipped: 1 DP, 1 SVM root, 1 not online*. It is worked out from the volume list loaded with **Load volumes**, so it costs no request to ONTAP. The skip rules need two optional fields (is_svm_root and nas.path). If an older ONTAP rejects them, the script asks again for the basic fields; SVM root volumes then cannot be recognised and are scanned, which is harmless because they are small.

What multi-volume scans add

- **Progress.** A line above the tiles reads Volume N of M with the current volume's name.
- **Failures do not stop the scan.** If one volume cannot be read (for example 403 or 404 on its root), the error is logged on the Errors tab with the volume's name and the scan moves to the next one. The scan as a whole fails only if no volume could be scanned.
- **Per-volume results.** The Summary tab has a By volume table (state, files checked, cold files, cold size, cold percentage), and the Charts tab a By volume bar chart. Clicking a volume opens the Files tab filtered to that volume.
- **Volume column, filter and sort.** The Files tab gets a Volume column, a Volume filter (text in svm:volume; start with = for an exact match) and a Volume sort option. The largest-cold-folders views show the volume.
- **Start folder.** Used only by the One volume scope. Other scopes start at each volume's root.

- **Scan time adds up.** Volumes are scanned one at a time, so a whole-cluster scan takes about as long as the volumes take one after another. Raise Workers carefully (see section 9).

4. From the GUI to the script

The GUI never talks to ONTAP. The browser talks only to the script, and the script talks to ONTAP. Every GUI action is a small JSON request to one of these routes.

Route	What it does	Calls ONTAP?
GET /	Serves the embedded page. Needs no token and contains no data.	No
GET /api/status	Counters, age buckets, largest folders, per-volume figures, skipped volumes, errors and the scan state. The page polls this every 1.5 seconds.	No
GET /api/results	One page of the Files table, with sort, search and filters applied (a database query).	No
GET /api/history	Lists the saved CSV files in the scans folder and whether each can be reopened.	No
GET /api/file?name=	Downloads one saved CSV.	No
POST /api/test	Cluster name and ONTAP version.	Yes, 1 request
POST /api/volumes	The volume list for the picker, the checklist and the preview, with the skip reason of each volume.	Yes, 1 or a few
POST /api/scan/start	Validates the settings and starts a scan.	Yes, many (the scan)
POST /api/scan/stop	Sets the stop flag so workers exit.	No
POST /api/scan/open	Reopens a saved scan from its database file. Refused while a scan is running.	No

What each button does

Button or action	Request	What the script does
Test connection	POST /api/test	Builds an ONTAP client from the form fields and calls GET /api/cluster. Shows the cluster name and version, or the error.
Load volumes	POST /api/volumes	Lists every volume (name, UUID, SVM, state, type, style) with its default skip reason. Fills the volume picker, the checklist and the SVM suggestions.
Start scan	POST /api/scan/start	Checks the scope and its inputs (a volume UUID, a list of UUIDs, or an SVM name), the criteria, and clamps the numbers. Creates a Scan and starts it.
Stop	POST /api/scan/stop	Sets a flag. Workers notice it between entries and exit. The CSV and database keep what was found so far.
Progress tiles, Summary, Charts	GET /api/status every 1.5 s	Copies the counters under the lock and returns them. No ONTAP calls.
Files tab: sort, search, filters, paging	GET /api/results	Runs one SQL query against the scan's database and returns 100 rows. No ONTAP calls.
Download CSV, Saved scans	GET /api/file	Sends the file from the scans folder. Only a bare file name ending in .csv is accepted.
Saved scans: Open	POST /api/scan/open	Loads the saved summary and points the Files tab at that scan's database. No ONTAP calls.

Sorting and filtering never rescan. They only reorder or narrow the rows the scan already stored. Only Start scan (and Test connection and Load volumes, once each) sends requests to ONTAP. In the terminal where the script runs you will see /api/status and /api/results lines when you use the page, and a POST /api/scan/start line only when a scan begins.

The results request

Parameter	Meaning
q	Path contains this text (not case sensitive).
sort	Up to four items like mtime:asc,size:desc. Keys: path, size, atime, mtime, age, volume.
offset, limit	Paging. Limit is 1 to 1000; the GUI uses 100.
fmin, fmax	Size range in bytes (the GUI converts from KiB, MiB or GiB).
mod, acc	Not modified for at least this many days; not accessed for at least this many days.
ext	Only these file types, for example .vmdk,.iso.
vol	Volume filter: text contained in svm:volume, or =svm:volume for exactly one volume.

5. How it connects to ONTAP

The script connects to the cluster management LIF (the cluster's management address) over HTTPS on port 443. It does not connect to each node. The cluster routes each request internally, and if the node hosting the LIF fails, ONTAP moves the LIF and the scan carries on. Always enter the cluster management address, not a node address.

The REST requests

Purpose	Request
Test connection	GET /api/cluster?fields=name,version
Volume list (Load volumes, and the start of every scan)	GET /api/storage/volumes?fields=name,uuid,svm.name,state,type,style,is_svm_root,nas.path&max_records=500 (falls back to the same without the last two fields on HTTP 400)
Check the root path (once per volume)	GET /api/storage/volumes/{uuid}/files/ then, if needed, .../files/%2F, with max_records=1
List one folder (once per folder)	GET /api/storage/volumes/{uuid}/files/{path}?fields=name,type,size,bytes_used,accessed_time,modified_time&max_records=1000

Authentication and TLS

- **Authentication:** HTTP basic authentication, with the user and password typed in the GUI. It is sent only over HTTPS.
- **Certificate check:** three choices. Verify with system CA store (default). Verify with CA file, using a path on the scan host, which suits a private internal CA. Skip verification, which turns off all certificate and host name checks and should not be used in production.

Details that matter

- **Path encoding.** The {path} part is a single path parameter, so slashes inside it are sent as %2F (for example projects%2Farchive_2022). The path is relative to the volume, not an SMB share path or an NFS export path.
- **Root path.** ONTAP versions can differ in how they accept the volume root, so the script tries an empty path and then %2F and keeps whichever works, separately for each volume.
- **Paging.** A folder with more than 1,000 entries comes back in pages. The script follows the _links.next.href link exactly as given until none is left.
- **Retries.** HTTP 429, 500, 502, 503 and 504, and network errors, are retried up to 4 times, waiting 1, 2, 4 and 8 seconds. Each request times out after 60 seconds. Other errors (such as 401, 403 and 404) fail that request at once.
- **Skipped entries.** The script skips ., .., .snapshot and ~snapshot. Scanning snapshot folders would multiply the work and give misleading ages. Only entries of type file are checked, and only entries of type directory are walked into. Symbolic links are not followed.
- **Size.** The size used is bytes_used (actual space used), falling back to size.
- **Timestamps.** Times are parsed in a way that accepts Z, +00:00 and +0000 endings, with or without fractions of a second.

> **One volume at a time.** The files API stays inside one volume, and a junction into another volume is not followed. The cluster and SVM scopes cover the other volumes by scanning each of them from its own root. A volume that is not mounted in the namespace may refuse the files API; it then shows as failed and the scan continues. This has not been checked against a real cluster.

6. How a scan works

Life of a scan

Step	What happens
1 Validate	The start route checks the settings and creates a Scan object that holds the settings, an ONTAP client and the counters. The time the scan starts is fixed, and every age is measured from it.
2 Plan	The script lists the cluster's volumes (one request) and decides which to scan from the scope and the skip rules. Skipped volumes are recorded with the reason. If nothing is left to scan, the scan fails with a message.
3 Open outputs	The CSV (header row first) and the SQLite database are created in the scans folder, readable by the owner only.
4 Each volume in turn	Probe the root (one small request that confirms the login, the volume UUID and the root path style), put the start folder in the queue, start the workers, wait for them to finish. If the probe fails, the volume is marked failed and the next one starts.
5 Finish	Commit the last rows and build the database indexes (the message "Building the search index..." shows). The final state is done, stopped (Stop pressed) or failed (no volume could be scanned). The summary is saved inside the database, the CSV is closed.

The worker loop (runs once per volume)

```
while not stop_flag:
    take a folder from the queue (wait 0.5 s if empty;
        exit if nothing is queued or in progress)
    list the folder through ONTAP (follow every page)
    subfolders -> collected to add to the queue
    files      -> excluded? counted apart. otherwise age-checked
    lock; write this folder's cold files to the database and CSV;
          update the totals (overall and for this volume);
          add subfolders to the pending count; unlock
    put the subfolders on the queue
```

Knowing when a volume is finished. An empty queue does not mean the volume is done, because another worker may be about to add subfolders. The scan keeps a counter of folders that are queued or being read. Workers exit only when that counter reaches zero. **Why a shared queue.** Workers take whatever folder is next, so one deep branch does not leave the others idle.

A failed folder does not stop the scan. If one folder returns an error (for example 403 permission denied), the error is recorded (Errors tab, with the volume name in multi-volume scans) and the scan moves on.

How a file is judged

Setting	Rule
Both modified and accessed are older (default)	The file's age is the number of whole days since the more recent of its two timestamps. It is cold if that age is at least the threshold.
Modified time is older	Only the modified time is used.
Accessed time is older	Only the accessed time is used.
Missing timestamp	If a timestamp needed for the chosen rule is missing, the file is not judged and is counted as having no usable timestamp.
Exclude file types	A file whose name ends with one of the listed endings (any case) is counted apart as excluded. It is not age-checked and not in the totals.

Access times can mislead. ONTAP updates access times at a coarse interval (about a day by default), and backup, antivirus and indexing tools can touch them so cold data looks warm. The default rule (both older) is the safer choice. Compare modified and accessed dates in the Files tab to see how much noise there is.

What is kept

- Totals: folders scanned, files checked, cold files, cold size, size of everything checked, excluded files and size. The same figures per volume.
- Age buckets: under 1 year, 1 to 2 years, 2 to 3 years, 3 to 5 years, over 5 years (by the file's age).
- Largest cold folders: cold size grouped by volume and the first N parts of the folder path (N is the folder depth setting). The top 30 are shown. A safety limit of 100,000 distinct groups lumps any further folders into one (other) group per volume; at the default depth it is never reached.
- File type and year totals for the Charts tab.
- Every cold file: in the database and the CSV. Nothing is held in memory and there is no row cap.

7. Results, the database, sorting, filters and the CSV

Where results live

Each scan creates two files in the scans folder, with the same base name, for example cryo_cluster_20261028-143012.csv and cryo_cluster_20261028-143012.db. The name carries the scan's label (the volume name, selected, the SVM name or cluster) and its start time. Both files hold file paths, so both are created readable by the owner only (where the operating system allows).

File	Holds	Used for
CSV	One row per cold file. Columns: svm, volume, path, bytes_used, accessed_time, modified_time, age_days. A text field that begins with =, +, - or @ gets a leading apostrophe, so spreadsheet programs do not treat it as a formula.	Download, spreadsheets, other tools
SQLite database	A files table (one row per cold file: volume, path, size, accessed and modified time, age, days since modified, days since accessed) and a meta table that stores the finished scan's summary.	The Files tab (sort, filter, paging) and reopening a saved scan

How the database is used

- **While the scan runs.** Workers write each folder's cold files in one batch. The database is in WAL mode with synchronous=NORMAL, which is safe and avoids a disk flush per commit. Changes are committed about once a second, so the Files tab shows new rows within a second or two.
- **When the scan ends.** The indexes on size, modified time, accessed time and age are built (this is the "Building the search index..." step), the summary is stored, and the database is switched back to a single plain file.
- **The Files tab.** Each page is one SQL query on a read-only connection. The sort column comes from a fixed list, every value is passed as a bound parameter, and the file id is added as a final tie-break so paging is stable. In a test with one million rows, a sorted page came back in under a quarter of a second; a query that combined a file-type filter, a size filter and a sort over almost all rows took about three seconds.
- **If the scan fails before finding anything** (for example a wrong password), the empty CSV and database are deleted.
- **Reopening a saved scan.** Saved scans lists each CSV. If its database is still next to it, an Open link loads the saved summary (tiles, Summary, Charts, Errors) and points the Files tab at that database. Nothing is sent to

ONTAP. It is refused while a scan is running. If you delete the .db file, the CSV can still be downloaded but not reopened.

A result row (from /api/results)

Position	Field	Notes
0	Path	Volume-relative, starting with /
1	Size	Bytes used
2	Accessed time	ISO date and time
3	Modified time	ISO date and time
4	Age (days)	The cold age used for buckets and the Age column
5, 6	Days since modified, days since accessed	Used by the "not modified for" and "not accessed for" filters
7	Volume	svm:volume, shown as a column in multi-volume scans

Sorting

The GUI offers up to three sort levels (a column and a direction for each). They become one ORDER BY clause: the first level, then the second among ties, then the third. Clicking a column header makes it the first level. The quick sort buttons fill in the levels:

Quick sort	Levels
Largest files	Size, largest first
Longest unmodified	Modified date, oldest first; then size, largest first
Least recently accessed	Accessed date, oldest first; then size, largest first
Coldest, then largest	Modified oldest first; accessed oldest first; then size, largest first

Filters

Filters narrow the stored rows and are combined, so every filter must match. The line under the table shows how many files match and their total size. Filters apply only to the Files tab: the Summary tab, the Charts tab and the CSV always cover the whole scan.

> The scan only saves files that were cold at the threshold you set. You can filter more strictly afterward (for example 730 days after scanning at 180), but you cannot loosen the threshold, the criteria or the exclusions without starting a new scan.

8. Security

What is built in

Area	What the script does
Read-only to ONTAP	Only GET requests are sent. The script cannot change, move or delete data.
Password handling	The password is sent from the browser to the script in the request body and used to build the authentication header. It is held in memory by the current scan object until the next scan replaces it or the program exits. It is never written to disk, the CSV or the database, and the status response does not include it.
TLS to ONTAP	Certificate verification is on by default. A CA file can be given for a private CA. Skipping verification is possible but not recommended.
GUI access token	With --token (or the CRYO_TOKEN variable), every API route needs the token in an X-Token header (or a token parameter for download links). The page asks for it once per browser session. The page itself is served without a token.
GUI over TLS	--certfile and --keyfile serve the GUI over HTTPS.
Local by default	The script listens on 127.0.0.1. It warns at start-up if you bind to another address without a token or without TLS.
Input checks	The scope must be one of four. A volume must be a UUID; a list of volumes at most 5,000 UUIDs; an SVM name only letters, digits, dot, dash and underscore (64 characters at most). The criteria must be one of three values. Numbers are clamped (days 1 to 36500, workers 1 to 16, depth 1 to 10). At most 50 excluded endings.
Result files	The CSV and the database are created readable by the owner only. Database queries use bound parameters, a fixed list of sort columns, and a read-only connection, so a crafted request cannot change or inject into the SQL.
File download and reopen safety	Only a bare file name ending in .csv, inside the scans folder, can be downloaded or reopened. Folder tricks like ../ are removed. The database name is derived from the CSV name.
Safe page output	File names, volume names and error text from ONTAP are escaped before being drawn, so a hostile name cannot run script in the page.
CSV safety	Text that starts with =, +, - or @ is prefixed so spreadsheets do not run it as a formula.
Responses	Sent with Cache-Control: no-store and X-Content-Type-Options: nosniff.
No outside traffic	No CDN, fonts, updates or telemetry. The script contacts only the cluster address typed into the GUI.

Known gaps (be aware of these)

- **One shared token, no user accounts.** There is no per-user login, no audit log of who started a scan, no limit on wrong guesses, and the token is compared with ordinary equality rather than a constant-time check.
- **Token in download links.** The download links carry the token in the address, so it can appear in the script's access log and in browser history.
- **Password crosses the network.** The password goes from your browser to the script. On plain HTTP over a network, it is readable. Use TLS or an SSH tunnel.
- **With no token set,** anyone who can reach the port can use the GUI, open any saved scan, and make the scan host send requests to any address they type. There are no Origin or Host checks.
- **Results are not encrypted.** The CSV and database files hold file paths and ages, which are sensitive. They are readable only by the account running the script, but sit unencrypted on disk. A cluster-wide scan stores far more paths than a single-volume scan.

- **The access log** is printed to the terminal, so nothing is retained unless you capture it.

Recommended production setup

1. **Least-privilege ONTAP account.** Create a REST role with read-only access to /api/cluster and /api/storage/volumes, and a service account that uses it. Limit which hosts can reach the management LIF to the scan host. Check the exact commands on your ONTAP version:

```
security login rest-role create -role cryo_ro -api /api/cluster -access readonly
security login rest-role create -role cryo_ro -api /api/storage/volumes -access readonly
security login create -user-or-group-name svc_cryo -application http \
      -authentication-method password -role cryo_ro
```

2. **Do not expose the script directly.** Bind it to 127.0.0.1 and put a reverse proxy (nginx or HAProxy) in front, with TLS from your internal CA and real user sign-in (SSO, or client certificates). Keep the token as a second layer.
3. **Always verify TLS to the cluster** with your internal CA file. Never use Skip verification.
4. **Run as a dedicated non-login user** with hardening (example below), SELinux enforcing, and a scans folder readable only by that user.
5. **Protect results.** Put the scans folder on an encrypted volume, restrict who can read it, and delete old CSV and database files on a schedule.
6. **Add an audit trail.** Send the script's output to syslog or your SIEM, and match it with ONTAP's own logs for the service account. A per-scan audit line (user, scope, settings, times) is a good next code change.
7. **Keep credentials off the browser.** A good next code change is to read the service account's password from a protected file on the server and remove the password box from the page.
8. **Control changes.** Keep the script in Git with review, and record the SHA-256 of the deployed file.

```
[Unit]
Description=Cryo cold data finder
After=network.target

[Service]
User=cryo
ExecStart=/usr/bin/python3 /opt/cryo/cryo.py \
    --host 127.0.0.1 --port 8080 --outdir /var/lib/cryo/scans
EnvironmentFile=/etc/cryo/token.env
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
ReadWritePaths=/var/lib/cryo
CapabilityBoundingSet=
UMask=0077
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
# nginx: TLS in front, proxy to the local script
server {
  listen 443 ssl;
  server_name cryo.example.local;
  ssl_certificate     /etc/pki/tls/certs/cryo.crt;
  ssl_certificate_key /etc/pki/tls/private/cryo.key;
  # add your SSO or client-certificate checks here
  location / { proxy_pass http://127.0.0.1:8080; proxy_read_timeout 120s; }
}
```

9. Running it and troubleshooting

Start it

```
python3 cryo.py                      # http://127.0.0.1:8080
python3 cryo.py --port 9000 --outdir /var/lib/cryo/scans
ssh -L 8080:127.0.0.1:8080 you@scanhost       # reach it from your workstation
```

Option	Meaning	Default
--host	Address to listen on	127.0.0.1
--port	Port to listen on	8080
--outdir	Folder for the CSV and database results	scans
--token	Access token for the API (or set CRYO_TOKEN)	none
--certfile, --keyfile	Serve the GUI over HTTPS	off

First real run

1. Enter the management address and a read-only account. Click Test connection. Then Load volumes.
2. Start small: scope One volume, a small volume, and a start folder with a few hundred files. Leave workers at 4 to 6.
3. Click Start scan. Watch the folders-per-second tile and the Errors tab.
4. Check a few results by hand against the files themselves, comparing the modified and accessed dates.
5. Then try Selected volumes with two or three volumes, then One SVM, and only then Whole cluster. Look at the preview line first, and at the By volume table afterward to see which volumes failed or were skipped.
6. If folders per second stops rising as you add workers, the cluster is the limit, so stop adding workers.

Choosing the number of workers

The work is waiting on the network, so threads are enough. Start at 4 to 6. More workers means more simultaneous requests on the management LIF, so watch cluster CPU while you test. Parallelism is across folders, not within one: a single folder with millions of files is read by one worker, page by page. In a multi-volume scan the workers cover one volume at a time, so a small volume may not use them all.

Troubleshooting

What you see	Likely cause and fix
HTTP 401	Wrong user or password.
HTTP 403	The account's role does not allow the API. Check the REST role. In a multi-volume scan this fails only that volume.
HTTP 404 for a volume	Wrong volume UUID, a start folder that does not exist (One volume scope), or a volume that is not mounted in the namespace. In a multi-volume scan the volume is marked failed and the scan continues.
No volumes found in SVM ...	The SVM name does not match any volume. Use Load volumes and pick the name from the suggestions.
No volumes to scan	Every volume was skipped by the default rules (or not found). Tick volumes by hand, or turn on Include DP volumes.
No volume could be scanned	Every volume failed. The Errors tab lists each one.
Certificate verify failed	The script does not trust the cluster's certificate. Use Verify with CA file and give the CA path.
Timeout or connection refused	The scan host cannot reach the management address on port 443. Check routing and firewall.
A scan is already running	Only one scan at a time. Press Stop, or wait for it to finish. Opening a saved scan is refused for the same reason.
Volume must be a UUID	Use Load volumes and pick from the list, or paste the volume's UUID.
That saved scan has no results database	The .db file next to the CSV was deleted or never created. The CSV can still be downloaded.
Folder errors on the Errors tab	Usually permissions or a folder removed during the scan. The scan continues and the rest of the results are still valid.
Cold counts look too low	The default rule needs both modified and accessed to be old. Try modified only, and check the excluded file types.
Lots of /api/status lines in the terminal	Normal. The page polls every 1.5 seconds.

10. Limits and known gaps

- **Speed.** One REST request per folder. A volume with tens of millions of files needs millions of requests and will be slow, and a whole-cluster scan is the sum of its volumes because they are scanned one at a time. Scan targeted subtrees or volumes, or consider a faster scan over an NFS mount.
- **Excluding file types does not speed up a scan.** ONTAP still returns every entry. Only skipping folders would save requests, and the tool does not have a folder-exclusion setting yet.
- **New connection per request.** Each request opens a new HTTPS connection. Reusing connections would speed up large scans.
- **Disk space.** Every cold file is stored twice (CSV and database), so a very large scan can need gigabytes. Keep the scans folder on a volume with room, and delete old scans you no longer need.
- **Stop is not instant.** Workers stop between entries, so a request in progress finishes first. Volumes not yet reached are left unscanned and show as waiting.
- **Results are not cached across scans.** Each Start scan walks the volumes again. Changing the age threshold, the criteria or the exclusions needs a new scan. Sorting and filtering do not. A saved scan can be reopened and browsed, but not extended.

- **A scan stopped by a crash** (power loss, kill) leaves a CSV and a database without a saved summary. The CSV is still usable; the Open link needs the summary and will refuse it.
- **Files tab limits.** Search with a leading wildcard (text anywhere in the path) scans the table, so filtering combined with sorting on tens of millions of rows can take several seconds.
- **Access times are coarse** and can be touched by other tools (see section 6).
- **Counts are as of the scan start.** Files changed during a long scan may be judged against slightly old information.
- **FlexGroup and unmounted volumes are unverified.** The skip rules follow the documented volume fields, but how the files API behaves for a FlexGroup or a volume with no junction path has not been checked on a real cluster.
- **Not yet tested on a real cluster.** Tested with a mock server, a large synthetic database and headless-browser checks only.

Appendix A. Settings and limits

Item	Value
Python	3.6 or newer, standard library only
Scopes	One volume, selected volumes (1 to 5,000), one SVM, whole cluster
Workers	1 to 16 (default 6), per volume
Age threshold	1 to 36,500 days (default 180); a file is cold at that many whole days or more
Folder depth for grouping	1 to 10 (default 2)
Excluded file endings	Up to 50
Entries per ONTAP page	1,000
Request timeout and retries	60 seconds; up to 4 retries with 1, 2, 4, 8 second waits
Result rows	Unlimited (stored in the database and CSV, not in memory)
Folder groups kept for the largest-folders views	100,000, then (other) per volume
Volumes listed in the status answer	100 with the most cold data (totals are always complete); 200 skipped volumes listed
Folder errors kept	500 (the GUI shows the latest 100; the total count is always shown)
Sort levels in a request	Up to 4 (the GUI offers 3)
Rows per page in the Files tab	100 (the API allows up to 1,000)
Database commit interval	About 1 second while a scan runs

Appendix B. Glossary

Term	Meaning
LIF	Logical interface: a network address on the cluster. The management LIF is the address the REST API answers on.
SVM	Storage virtual machine: the unit that serves a set of volumes to clients.
SVM root volume	The volume that holds an SVM's namespace. Skipped by default.
DP volume	A data-protection volume: a SnapMirror or SnapVault destination, normally a read-only copy of another volume.
FlexGroup	A volume made of many constituent volumes spread over the cluster. The FlexGroup is scanned; its constituents are skipped.
Volume UUID	The unique ID of a volume, used in every files request.
Junction	The place where one volume is mounted inside another volume's namespace.
FSA	File System Analytics, an ONTAP feature that indexes files in the background. This tool does not need it.
atime, mtime	Last accessed time and last modified time of a file.
Cold data	Files not modified or accessed for at least the threshold you chose.
WAL	Write-ahead logging, the SQLite mode that lets the Files tab read while a scan is writing.
Basic authentication	Sending the user and password in an HTTP header. Safe only over TLS.

Appendix C. Charts tab

The Charts tab draws its charts from the summary of the current scan. The charts are plain SVG drawn by the page's own JavaScript, so they need no chart library and work in an air-gapped environment. They send nothing to ONTAP and need no rescan.

Chart	What it shows	Data it uses
Cold versus the rest	Cold size as a share of all scanned size, as a ring with the percentage in the middle.	cold_bytes and total_bytes
Cold data by age	Cold size in each age bucket (under 1 year up to over 5 years), as vertical bars.	The age buckets
Where the cold data is	A treemap: each box is a folder, and its area is its cold size. The largest 30 folders are shown, plus one grey box for everything else. In multi-volume scans a box is labelled with its volume. Clicking a box opens the Files tab filtered to that folder (and volume).	The largest cold folders, grouped at the folder depth setting
By file type	The ten biggest file types by cold size, plus an Other bar, with file counts. Clicking a bar opens the Files tab filtered to that type.	A per-extension total kept during the scan
By year last modified	Cold size by the year each file was last modified, with an unknown group for files without a usable time.	A per-year total kept during the scan
By volume (multi-volume scans only)	Cold size of the 15 volumes with the most cold data, as horizontal bars. Clicking a bar opens the Files tab filtered to exactly that volume.	The per-volume figures

How the file type is decided

The type is the ending of the file name after the last dot, in lower case. The endings .tar.gz, .tar.bz2 and .tar.xz are treated as one type. A name with no dot, or only a leading dot such as .bashrc, is counted as (none).

What the charts cover

- The charts cover the whole scan. They do not follow the filters on the Files tab.
- Files excluded by type are not in any chart, in line with the rest of the results.
- The folder treemap uses the 30 largest cold folders at the chosen depth. Raising the depth setting before a scan gives a finer map. Boxes that are too small for a label show the folder name and size when you point at them.
- Charts update while a scan runs, so early charts show partial results.